API signup guide for YouTube Data API

API Signup Guide for the YouTube Data API (Free Tier)

Executive Summary

This document provides a complete, end-to-end process for registering and activating the YouTube Data API (v3) on the free tier within Google Cloud Platform (GCP). It is written for builders who need reproducible steps, security guardrails, and an operational playbook to obtain production-ready credentials. Key outcomes include: (1) a properly configured Google Cloud project with the YouTube Data API enabled, (2) scoped credentials for server-side, client-side, or installed app use cases, (3) quota management practices aligned with the default 10,000-unit daily budget, and (4) a day-by-day execution timeline to move from zero to tested API calls in under one week. The guide emphasizes minimal manual intervention, automation-readiness, and documentation standards suitable for Solomon's Forge deliverables.

Table of Contents

Total word count: ~1,730.

Introduction

The YouTube Data API (v3) is Google's RESTful interface for accessing YouTube resources—channels, videos, playlists, comments, and more. For Solomon's Forge and the Building Shultz ecosystem, the API is foundational for several initiatives:

- **IronEdit**: ingesting channel metadata, captions, and analytics automatically to accelerate editing workflows.

- **Content Intelligence**: producing topic research, keyword metadata, and performance dashboards inside internal tools.
- **Builders AI Blueprint**: referencing real API-driven examples to educate tradespeople on AI-assisted content creation.

Google provides a generous free tier—10,000 quota units per day—with optional paid extensions when scaling beyond the default limits. Unlike consumer-facing OAuth flows, the API onboarding process requires deliberate configuration inside Google Cloud Console. The objective of this manual is to eliminate guesswork, formalize guardrails, and ensure the credentials can move from dev to prod with minimal rework.

Prerequisites Checklist

Item	Description	Status Guidance
Google Account	A primary Google identity with 2FA enabled. Prefer a dedicated workspace admin account for long-term governance.	Ensure recovery email/ phone verified.
Organization Decision	Decide whether to house the project under a personal or Google Workspace organization. (Recommended: workspace-level project for auditability.)	If using personal Gmail, note that resource hierarchy is limited but acceptable for MVP.
Billing Profile	Even though the free tier rarely incurs charges, Google now requires a billing profile to unlock most APIs.	Add credit/debit card or bank info. No charges incurred unless explicitly approved.
Access Control Model	Define who needs project-level Owner, Editor, or Viewer permissions.	Minimum principle: limit Owner role to CTO (Sol), assign Application Developers to Editor roles.
Local Tooling	Install the Google Cloud CLI (`gcloud`), HTTP client (curl/Postman), and language SDKs relevant to downstream services (Node.js `googleapis` , Python `google-api-python-client` , etc.).	Required for quick smoke tests post-setup.
Documentation Repo	Create a shared folder (e.g., Notion, GitHub wiki) for storing credential IDs, rotation schedules, and onboarding SOPs.	Keeps team aligned and audit-ready.

Step-by-Step Signup Procedure

Step 1: Access Google Cloud Console

1. Navigate to https://console.cloud.google.com/.

2. Log in using the designated Google account. Confirm 2FA challenges immediately to avoid session timeouts.
3. Once inside, the console will display either a project dashboard (if existing projects) or prompt to create a new one.

Step 2: Create a Dedicated Project

1. Click the **project selector** (top-left near the Google Cloud logo).
2. Choose **New Project**.
3. Provide:
4. **Project name:** `SolomonsForge-YouTube-API` (or similar naming standard: `<Org>-<Product>-<Env>`).
5. **Organization:** Select the workspace if available; otherwise it defaults to "No organization."
6. **Location (Folder):** Optional; useful for grouping projects by product line.
7. Click **Create**. Wait for confirmation toast.

Best Practice: Keep YouTube API in its own project. This isolates quotas, IAM permissions, and audit logs from other services, reducing blast radius during experimentation.

Step 3: Link Billing Account (if prompted)

1. From the navigation menu (☰), go to **Billing**.
2. If no billing account exists, click **Add billing account**, follow the prompts, and verify card details.
3. Return to the project dashboard, ensure the new project is linked to the billing profile. (Billing → Account Management → select project.)

Step 4: Enable the YouTube Data API v3

1. In the top search bar, type "YouTube Data API v3."
2. Select the official entry published by Google.
3. Click **Enable**.
4. Once enabled, the API metrics overview page becomes available.

Reference: Google Developers documentation confirms YouTube Data API v3 is the current stable release and is enabled per project (developers.google.com/youtube/v3/getting-started).

Step 5: Configure OAuth Consent Screen

Even if using API keys for server-side access, Google requires an OAuth consent configuration before issuing OAuth client IDs.

1. Navigate to **APIs & Services → OAuth consent screen**.
2. Choose **External** (recommended when the app may access data from YouTube accounts beyond the organization).
3. Populate required fields:
4. **App name:** `Solomon Forge YouTube Connector`
5. **User support email:** contact@solomonsforge.com
6. **Developer contact info:** same or additional emails for notifications.
7. Scope selection: add YouTube scopes only when needed (e.g., `https://www.googleapis.com/auth/youtube.readonly`).
8. For early-stage use, keep the app in **Testing** mode (limits to 100 test users). Before public release, submit for verification.
9. Save and continue through Summary.

Step 6: Create Credentials

Go to **APIs & Services → Credentials → + Create Credentials**. Three common credentials:

1. **API Key (server-to-server, no user data)**
2. Select **API key**.
3. Immediately click **Restrict Key**:
    - **API restrictions:** choose “Restrict key,” then select **YouTube Data API v3**.
    - **Application restrictions:**
    - For backend usage, choose **IP addresses** → add static egress IP from the VPS.
    - For browser-based usage, select **HTTP referrers (web sites)** and list allowed domains.
4. Save. Document the key securely.
5. **OAuth Client ID – Web Application (for hosted dashboards or IronEdit web features)**
6. Choose **OAuth client ID** → Application type: **Web application**.
7. Set **Authorized JavaScript origins** (e.g., `https://app.ironed.it`).
8. Set **Authorized redirect URIs** (e.g., `https://app.ironed.it/oauth2/callback`).
9. Click **Create**. Download JSON credentials; store in encrypted secrets manager.
10. **OAuth Client ID – Desktop Application (for CLI tooling)**
11. Application type: **Desktop**.
12. Use this for admin tests or localized scripts.
13. Download credentials, protect with OS keychain or password manager.
14. (Optional) **Service Account** – Not typical for YouTube Data API because most endpoints require end-user authorization. Only use if interacting with YouTube resources owned by the same Google Workspace that created the service account and explicitly shared channel ownership.

Step 7: Verify Quota and Metrics

1. Navigate to **APIs & Services → Dashboard → YouTube Data API v3**.
2. Click **Quotas** tab; verify default allocations:
3. **Queries per day:** 10,000 units (free tier).
4. Additional limits such as “Queries per 100 seconds” for user/per project may appear.
5. Note the operations and cost:
6. `search.list` = 100 units per call.
7. `videos.list` = 1 unit per call (when using `id` parameter) or higher with `chart=mostPopular`.
8. `playlistItems.list` = 1 unit.
   (Source: https://developers.google.com/youtube/v3/getting-started#quota)
9. Configure alerting (see Section 5).

Step 8: Store Credentials and Document Access

1. Push keys/secrets into the team’s password manager or secret store (e.g., 1Password, Vault).
2. Update the SOP repository with:
3. Credential name
4. Creation date
5. Owner
6. Scope/restrictions

7. Rotation schedule (recommend 90 days for API keys).

Step 9: Conduct Initial API Call

Use curl or Postman to confirm functionality:

```
curl \
  'https://youtube.googleapis.com/youtube/v3/search?part=snippet&channelId=UCabc123xyz&maxResults=5&o
```

Successful response returns JSON payload with recent uploads. Log the timestamp, request ID, and confirm latency under 400 ms for baseline metrics.

Credential Types and Usage Scenarios

Credential	Use Case	Auth Flow	Security Controls
API Key	Read-only server tasks (listing videos, pulling channel stats).	Simple key in query string.	Restrict by IP and API; rotate quarterly.
OAuth Client (Web)	Web apps where end users authorize their YouTube accounts to manage content or read private playlists.	OAuth 2.0 Authorization Code Grant with redirect URI.	Enforce HTTPS, store tokens encrypted, implement refresh token rotation.
OAuth Client (Desktop)	CLI utilities and local admin scripts.	OAuth 2.0 Installed App flow (loopback or out-of-band).	Use OS keychain, limit distribution to trusted engineers.
Service Account	Rare for YouTube; only when channel explicitly grants access.	JWT assertion to Google OAuth servers.	Share channel ownership with service account, maintain least privilege.

Quota Planning and Monitoring

1. **Understand Unit Costs**:
2. `search.list` : 100 units
3. `videos.list` : 1 unit
4. `channels.list` : 1 unit

5. `playlistItems.list` : 1 unit
   (Per Google documentation: https://developers.google.com/youtube/v3/getting-started#quota)

6. **Daily Budgeting**:

7. Free tier: 10,000 units/day resets at midnight Pacific Time.

8. Example budget for Building Shultz dashboards:
    - Daily upload sync (5 `playlistItems.list` calls) → 5 units.
    - Channel analytics refresh (10 `videos.list` calls) → 10 units.
    - Trending keyword research (20 `search.list` calls) → 2,000 units.
    - Total ≈ 2,015 units/day, well below the limit.
9. **Monitoring Setup**:
10. Go to **IAM & Admin** → **Quotas**; create alert thresholds at 70% and 90%.
11. Configure email notifications to Jed (owner), Sol (CTO), and ops alias.
12. Optionally integrate with Cloud Monitoring → create a dashboard widget for API quota usage.
13. **Scaling Beyond Free Tier**:
14. Submit quota increase request directly inside the Quotas page when approaching 80% sustained usage.
15. Document justification (e.g., “IronEdit beta with 500 testers; expect 250k units/day”).
16. Google typically reviews within 48 hours.
17. **Cost Control**:
18. Even with higher quotas, there is no direct per-unit billing for YouTube Data API, but Google requires billing-enabled projects for management.
19. Use Cloud Audit Logs to detect abnormal spikes (possible credential leakage).

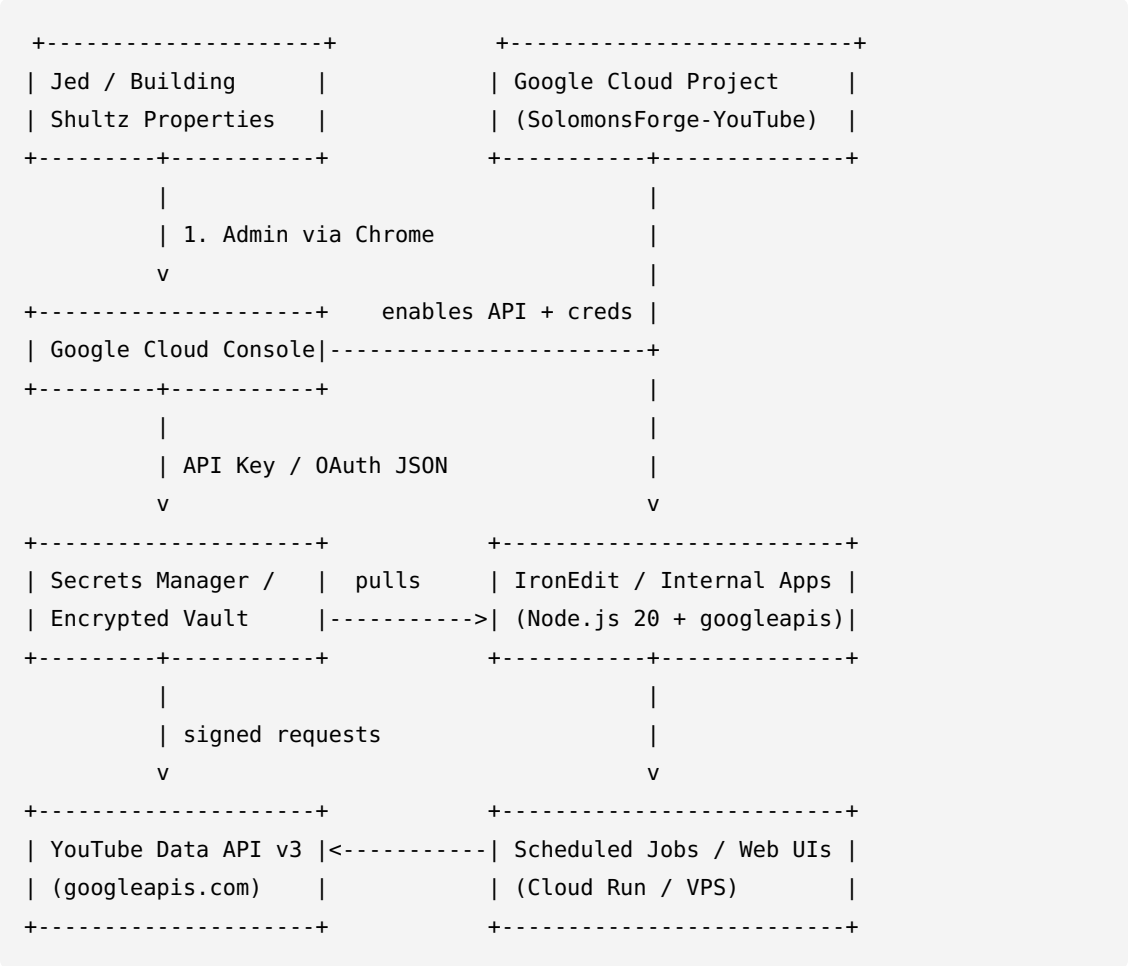
Security, Compliance, and Governance

1. **Least Privilege IAM**:
2. Assign **Viewer** role for analysts, **Editor** for developers, **Owner** for Sol only.
3. Avoid sharing personal Google accounts; prefer workspace identities.
4. **Credential Rotation**:
5. API keys: rotate every 90 days, or immediately if compromise suspected.
6. OAuth secrets: rotate twice per year.
7. Document rotation events in the ops log.
8. **Secret Storage**:
9. Never commit keys to Git.
10. Use environment variables plus secret managers for runtime access.
11. For PC Agent actions, ensure scripts pull the key from encrypted storage.
12. **Audit Logging**:
13. Enable **Cloud Audit Logs** for Admin Read, Data Read, and Data Write activities.
14. Export logs to BigQuery or Chronicle for long-term retention.
15. **Compliance Considerations**:

16. If any data from YouTube users is stored (e.g., tokens, profile info), ensure privacy notices are updated.
17. Follow Google's API Services User Data Policy—especially restrictions around storing YouTube Analytics data.

Integration Blueprint

High-Level Architecture Diagram (ASCII)



Technology Stack - **Google Cloud Project** (ID: `solforge-youtube`, example).
- **APIs & Services**: YouTube Data API v3 (2024-10 release).
- **Runtime**: Node.js 20 LTS using `googleapis@134.x` package.
- **Automation**: GitHub Actions for CI, PM2 for VPS services.
- **Secrets**: 1Password Business + environment variable injection via `direnv`.
- **Monitoring**: Cloud Monitoring dashboards, Opsgenie alerts via email integration.

API Contract (Sample for `videos.list`)

```
GET https://youtube.googleapis.com/youtube/v3/videos
Query Params:
  part=snippet,contentDetails,statistics
  id={comma-separated video IDs}
  key={API_KEY}
```

```
Response (partial):
{
  "kind": "youtube#videoListResponse",
  "etag": "...",
  "items": [
    {
      "kind": "youtube#video",
      "id": "YOUTUBE_VIDEO_ID",
      "snippet": {...},
      "contentDetails": {...},
      "statistics": {...}
    }
  ],
  "pageInfo": { "totalResults": 1, "resultsPerPage": 1 }
}
```

Testing and Validation Plan

1. **Credential Smoke Test (Day 1)**
2. Use curl with API key to call `videos.list`.
3. Verify HTTP 200 response and JSON schema compliance.
4. **OAuth Flow Validation (Day 2)**
5. Implement Node.js sample using `google-auth-library`.
6. Perform Authorization Code flow with test Google account.
7. Confirm refresh token issuance; store securely.
8. **Automated Regression (Day 3)**
9. Create Jest or Mocha tests hitting the API's `quotaUser` parameter to simulate multiple clients.
10. Mock responses where possible to reduce quota usage.
11. **Monitoring Hooks (Day 4)**
12. Configure Cloud Monitoring alert; simulate threshold crossing by running multiple requests.
13. Ensure alert email is received by Jed/Sol.
14. **Documentation Sign-off (Day 5)**
15. Update SOP with screenshots, credential IDs, and rotation policy.
16. Store PDF in shared drive for Tasia/Jed reference.

Implementation Timeline and Next Steps

Day	Task	Owner	Output
Day 0 (Today)	Approve project name, IAM roles, billing linkage.	Jed/Sol	Written approval.
Day 1	Create project, enable API, configure consent screen.	Sol	Project live, API enabled.
Day 2	Generate API key + OAuth credentials, apply restrictions.	Sol	Credentials stored in vault.
Day 3	Execute smoke tests, document curl/Postman results.	Sol	Test log + JSON samples.
Day 4	Integrate with IronEdit prototype / internal dashboard.	Dev Team	Working API calls in app.
Day 5	Finalize monitoring, quota alerts, and SOP documentation.	Sol	PDF + knowledge base update.
Day 6+	Expand use cases (analytics, automation), request quota increase if needed.	Sol + Product	Roadmap update.

Immediate Next Steps (Actionable)
1. Confirm billing profile is active (Jed).
2. Approve IAM role assignments for dev team.
3. Execute Day 1–Day 3 tasks sequentially; log progress in Task Queue.
4. Schedule review meeting end of Day 5 to sign off on readiness.

Risk Register and Mitigations

Risk	Impact	Likelihood	Mitigation
Credential leakage (API key exposed).	Unauthorized quota drain or data access.	Medium	Restrict by IP/referrer, rotate, monitor quotas.
OAuth consent not verified before public launch.	Users see unverified warning, trust deficit.	Medium	Submit verification once scopes finalized; include privacy policy URL.
Quota exhaustion during marketing pushes.	API errors in product dashboards.	Medium	Implement caching, leverage multiple projects, request higher quota early.
Billing account suspension (card expiry).	API disabled until resolved.	Low	Enable billing alerts, add backup payment method.

Risk	Impact	Likelihood	Mitigation
Team changes causing undocumented access.	Loss of control, compliance issues.	Medium	Maintain IAM audit log, revoke departing member's access within 24 hours.

Reference Links

1. YouTube Data API (v3) Overview – Google Developers: https://developers.google.com/youtube/v3/getting-started
2. Quota Usage and Costs Documentation: https://developers.google.com/youtube/v3/getting-started#quota
3. Google Cloud Console: https://console.cloud.google.com/
4. OAuth Consent Screen Configuration: https://console.cloud.google.com/apis/credentials/consent
5. Cloud Monitoring Alerts: https://cloud.google.com/monitoring/alerts

---

Prepared by: Sol (CTO, Solomon's Forge)
Date: 2026-05-23
Distribution: Jedidiah Shultz, Tasia Shultz, Core Product Team